

Realizar una investigación sobre el funcionamiento de los algoritmos de ordenamiento:

La investigación de cada algoritmo deberá contener:

- Algoritmo
- Aplicación paso a paso en un conjunto de 10 números enteros desordenados (no es necesario utilizar gráficas como el video, con mostrar cómo se mueven dentro de un arreglo es suficiente)

Insertion sort (ordenamiento por inserción)

Es un algoritmo simple de ordenamiento, eficiente sólo para ordenar arreglos pequeños. Su velocidad de procesamiento es inversamente proporcional al tamaño del arreglo: entre mayor es éste, la velocidad del algoritmo se hace cada vez más lenta. Es por ello que en sistemas complejos se prefiere utilizar algoritmos más avanzados.

Insertion Sort va construyendo un trozo ordenado del arreglo al extremo izquierdo, y en cada iteración del programa, le agrega un nuevo elemento a ese grupo. De este modo, se va formando un sub-arreglo (o sublista) ordenado al lado izquierdo, el cual crece 1 elemento en cada repetición, de modo que al llegar al final del arreglo (o lista), éste estará totalmente ordenado.

Para realizar el algoritmo el programa utiliza una variable auxiliar, la cual almacena el elemento que está siendo insertado. Se recorre la sublista de atrás hacia adelante para insertar el elemento en su lugar y así quedar ordenado respecto a los elementos que se encuentran antes en la lista, los cuales fueron ordenados en el paso anterior.

Una ventaja de Insertion Sort es que los requerimientos de memoria adicionales para el algoritmo son mínimos, ya que sólo utiliza la variable auxiliar antes nombrada: no se necesita de un arreglo adicional y además es fácil de implementar.

Algoritmo

```
#include <iostream>
using namespace std;

int main(){
    int numero[] = {4,2,3,1,5}; //arreglo a ordenar
    int i,pos,aux; //variables iniciales
    for (i = 0; i < 5; i++)// inicio de ciclo
    {
        pos = i; //posicion actual
        aux = numero[i]; //valor actual del arreglo
```

```

    while ( (pos>0) && (numero[pos1]) > aux) //mientras pos > 0 y el
valor a la izquierda es mayor al actual
    {
        numero[pos] = numero[pos-
1]; //se realiza el cambio de elementos

        pos--; // se reduce una posicion
    }
    numero[pos] = aux; //el valor actual es igual a valor actual
}

//se imprime el arreglo
cout<<"orden: "<<endl;
for (i = 0; i < 5; i++)
{
    cout<<numero[i]<<endl;
}
return 0;
}

```

Aplicación:

1. Nos colocamos en la primera posición del arreglo y verificamos si no hay elementos a su izquierda, como no hay pasamos a la siguiente posición.

5	2	3	4	6	7	9	8	1	10
---	---	---	---	---	---	---	---	---	----

2. Nos colocamos en la segunda posición y verificamos si hay elemento a la izquierda, si $5 > 2$ se realiza el cambio de elementos y avanzamos una posición.

5	2	3	4	6	7	9	8	1	10
---	---	---	---	---	---	---	---	---	----

Ahora verificamos si hay numero a la izquierda de 2. Pero no hay

2	5	3	4	6	7	9	8	1	10
---	---	---	---	---	---	---	---	---	----

Avanzamos a la siguiente posición y verificamos si hay un numero a su izquierda y si $5 > 3$

2	5	3	4	6	7	9	8	1	10
---	---	---	---	---	---	---	---	---	----

Entonces se realiza un cambio, verificamos si $2 > 3$, no lo es y avanzamos una posición

2	3	5	4	6	7	9	8	1	10
---	---	---	---	---	---	---	---	---	----

Verificamos si en la posición actual hay un numero a su izquierda y si $5 > 4$,

2	3	5	4	6	7	9	8	1	10
---	---	---	---	---	---	---	---	---	----

Entonces se realiza un cambio, verificamos si $3 > 4$, no lo es y avanzamos una posición

2	3	4	5	6	7	9	8	1	10
---	---	---	---	---	---	---	---	---	----

Verificamos si en la posición actual hay un numero a su izquierda y si $5 > 6$, no lo es, avanzamos una posición.

2	3	4	5	6	7	9	8	1	10
---	---	---	---	---	---	---	---	---	----

Verificamos si en la posición actual hay un numero a su izquierda y si $6 > 7$, no lo es, avanzamos una posición.

2	3	4	5	6	7	9	8	1	10
---	---	---	---	---	---	---	---	---	----

Verificamos si en la posición actual hay un numero a su izquierda y si $7 > 9$, no lo es, avanzamos una posición.

2	3	4	5	6	7	9	8	1	10
---	---	---	---	---	---	---	---	---	----

Verificamos si en la posición actual hay un numero a su izquierda y si $9 > 8$,

2	3	4	5	6	7	9	8	1	10
---	---	---	---	---	---	---	---	---	----

Lo es, hacemos el cambio y verificamos si $7 > 8$, no lo es y avanzamos una posición

2	3	4	5	6	7	8	9	1	10
---	---	---	---	---	---	---	---	---	----

Verificamos si en la posición actual hay un numero a su izquierda y si $9 > 1$

2	3	4	5	6	7	8	9	1	10
---	---	---	---	---	---	---	---	---	----

Hacemos el cambio y verificamos si hay número a la izquierda y si $8 > 1$, lo es y hacemos el cambio

2	3	4	5	6	7	8	1	9	10
---	---	---	---	---	---	---	---	---	----

verificamos si hay número a la izquierda y si $7 > 1$, lo es y hacemos el cambio

2	3	4	5	6	7	1	8	9	10
---	---	---	---	---	---	---	---	---	----

verificamos si hay número a la izquierda y si $6 > 1$, lo es y hacemos el cambio

2	3	4	5	6	1	7	8	9	10
---	---	---	---	---	---	---	---	---	----

verificamos si hay número a la izquierda y si $5 > 1$, lo es y hacemos el cambio

2	3	4	5	1	6	7	8	9	10
---	---	---	---	---	---	---	---	---	----

verificamos si hay número a la izquierda y si $4 > 1$, lo es y hacemos el cambio

2	3	4	1	5	6	7	8	9	10
---	---	---	---	---	---	---	---	---	----

verificamos si hay número a la izquierda y si $3 > 1$, lo es y hacemos el cambio

2	3	1	4	5	6	7	8	9	10
---	---	---	---	---	---	---	---	---	----

verificamos si hay número a la izquierda y si $2 > 1$, lo es y hacemos el cambio

2	1	3	4	5	6	7	8	9	10
---	---	---	---	---	---	---	---	---	----

verificamos si hay número a la izquierda, ya no hay

1	2	3	4	5	6	7	8	9	10
---	---	---	---	---	---	---	---	---	----

y avanzamos una posición en el arreglo

1	2	3	4	5	6	7	8	9	10
---	---	---	---	---	---	---	---	---	----

verificamos si hay número a la izquierda y si $9 > 10$, no lo es avanzamos una posición mas pero se acaba el arreglo y termina el ciclo

1	2	3	4	5	6	7	8	9	10
---	---	---	---	---	---	---	---	---	----

!!!ARREGLO FINAL, ACOMODADO!!!

1	2	3	4	5	6	7	8	9	10
---	---	---	---	---	---	---	---	---	----

Shell Sort

El ordenamiento de Shell, a veces llamado “ordenamiento de incremento decreciente”, mejora el ordenamiento por inserción al romper la lista original en varias sublistas más pequeñas, cada una de las cuales se ordena mediante un ordenamiento por inserción. La manera única en que se eligen estas sublistas es la clave del ordenamiento de Shell. En lugar de dividir la lista en sublistas de ítems contiguos, el ordenamiento de Shell usa un incremento i , a veces denominado brecha, para crear una sublista eligiendo todos los ítems que están separados por i ítems.

Algoritmo de ordenamiento Shell: El método se denomina así en honor de su inventor Donald Shell. Su implementación original, requiere $O(n^2)$ comparaciones e intercambios en el peor caso, aunque un cambio menor presentado en el libro de V. Pratt produce una implementación con un rendimiento de $O(n \log^2 n)$ en el peor caso. Esto es mejor que las $O(n^2)$ comparaciones requeridas por algoritmos simples pero peor que el óptimo $O(n \log n)$.

El Shell sort es una generalización del ordenamiento por inserción, teniendo en cuenta dos observaciones: El ordenamiento por inserción es eficiente si la entrada está "casi ordenada". El ordenamiento por inserción es ineficiente, en general, porque mueve los valores sólo una posición cada vez.

El algoritmo Shell sort mejora el ordenamiento por inserción comparando elementos separados por un espacio de varias posiciones. Esto permite que un elemento haga "pasos más grandes" hacia su posición esperada. Los pasos múltiples sobre los datos se hacen con tamaños de espacio cada vez más pequeños. El último paso del Shell sort es un simple ordenamiento por inserción, pero para entonces, ya está garantizado que los datos del vector están casi ordenados.

Algoritmo:

```
#include <iostream>

using namespace std;

void shell(int array[], int size)
{
    int i, j, intervalo, temp;

    intervalo = size / 2;

    while (intervalo > 0)
    {
        for (i = intervalo; i < size; i++)
        {
            j = i;
```

```

        temp = array[i];

        while ((j >= intervalo) && (array[j - intervalo] > temp))
        {

            array[j] = array[j - intervalo];

            j = j - intervalo;

        }

        array[j] = temp;
    }

    intervalo /= 2;
}
}

int main(int argc, char *const argv[])
{

    int numeros[] = {45, 27, 53, 17, 73, 56, 49, 19, 31, 34, 75, 45, 67, 21,
34, 12};

    /* Se muestra el arreglo original */

    for (int i = 0; i < 16; ++i)
    {

        cout << numeros[i] << " ";

    }

    cout << endl;

    shell(numeros, 16);

    /* Se muestra el arreglo ordenado */

    for (int i = 0; i < 16; ++i)
    {

        cout << numeros[i] << " ";

    }

    cout << endl;

}

```

Aplicación:

5	2	3	4	6	7	9	8	1	10
---	---	---	---	---	---	---	---	---	----

Se toma el tamaño del arreglo y se divide en 2; $10/2=4.5$ se toma el entero 4, si se subdividen en pareja de 2 en distancia de 4

5	2	3	4	6	7	9	8	1	10
---	---	---	---	---	---	---	---	---	----

Se realizan las comparaciones entre las parejas

- $5 < 7 = \text{si}$
- $2 < 9 = \text{si}$
- $3 < 8 = \text{si}$
- $4 < 1 = \text{no}$
- $6 < 10 = \text{si}$

5	2	3	1	6	7	9	8	4	10
---	---	---	---	---	---	---	---	---	----

Una vez que todos están acomodados se hace comparaciones en distancia de 2

- $5 < 7 = \text{si}$
- $2 < 9 = \text{si}$
- $3 < 8 = \text{si}$
- $1 < 4 = \text{si}$
- $6 < 10 = \text{si}$

5	2	3	1	6	7	9	8	4	10
---	---	---	---	---	---	---	---	---	----

- $5 < 3 = \text{no}$, se hace cambio 3 | 5
- $5 < 6 = \text{si}$
- $6 < 9 = \text{si}$
- $9 < 4 = \text{no}$, se hace cambio 4 | 9

3	2	5	1	6	7	4	8	9	10
---	---	---	---	---	---	---	---	---	----

- $2 < 1 = \text{no}$, se hace cambio 1 | 2
- $2 < 7 = \text{si}$
- $7 < 8 = \text{si}$
- $8 < 10 = \text{si}$

3	1	5	2	6	7	4	8	9	10
---	---	---	---	---	---	---	---	---	----

- $3 < 5 = \text{si}$
- $5 < 6 = \text{si}$
- $6 < 4 = \text{no}$, se hace cambio $4 | 6$
- $6 < 10 = \text{si}$

3	1	5	2	4	7	6	8	9	10
---	---	---	---	---	---	---	---	---	----

- $1 < 2 = \text{si}$
- $2 < 7 = \text{si}$
- $7 < 8 = \text{si}$
- $8 < 10 = \text{si}$

3	1	5	2	4	7	6	8	9	10
---	---	---	---	---	---	---	---	---	----

- $3 < 5 = \text{si}$
- $5 < 4 = \text{no}$, se hace cambio $4 | 5$
- $5 < 6 = \text{si}$
- $6 < 9 = \text{si}$

3	1	4	2	5	7	6	8	9	10
---	---	---	---	---	---	---	---	---	----

- $1 < 2 = \text{si}$
- $2 < 7 = \text{si}$
- $7 < 8 = \text{si}$
- $8 < 10 = \text{si}$

3	1	4	2	5	7	6	8	9	10
---	---	---	---	---	---	---	---	---	----

- $3 < 4 = \text{si}$
- $4 < 5 = \text{si}$
- $5 < 6 = \text{si}$
- $6 < 9 = \text{si}$

3	1	4	2	5	7	6	8	9	10
---	---	---	---	---	---	---	---	---	----

- $3 < 1 = \text{no}$, se hace cambio $1 \mid \mid 3$
- $3 < 4 = \text{si}$
- $4 < 2 = \text{no}$, se hace cambio $2 \mid \mid 4$
- $4 < 5 = \text{si}$
- $5 < 7 = \text{si}$
- $7 < 6 = \text{no}$, se hace cambio $6 \mid \mid 7$
- $7 < 8 = \text{si}$
- $8 < 9 = \text{si}$
- $9 < 10 = \text{si}$

1	3	2	4	5	6	7	8	9	10
---	---	---	---	---	---	---	---	---	----

- $1 < 3 = \text{si}$
- $3 < 2 = \text{no}$, se hace cambio $2 \mid \mid 3$
- $3 < 4 = \text{si}$
- $4 < 5 = \text{si}$
- $5 < 7 = \text{si}$
- $7 < 8 = \text{si}$
- $7 < 8 = \text{si}$
- $8 < 9 = \text{si}$
- $9 < 10 = \text{si}$
-

1	2	3	4	5	6	7	8	9	10
---	---	---	---	---	---	---	---	---	----

iii Arreglo final, acomodado!!!

Bibliografía

[https://wiki.dcc.uchile.cl/alice/doku.php?id=2_4#:~:text=explicar%20este%20%C3%BAltimo.-,Insertion%20Sort%20\(Ordenamiento%20por%20Inserci%C3%B3n\)%3A,hace%20cada%20vez%20m%C3%A1s%20lenta.](https://wiki.dcc.uchile.cl/alice/doku.php?id=2_4#:~:text=explicar%20este%20%C3%BAltimo.-,Insertion%20Sort%20(Ordenamiento%20por%20Inserci%C3%B3n)%3A,hace%20cada%20vez%20m%C3%A1s%20lenta.)

<https://runestone.academy/runestone/static/pythoned/SortSearch/ElOrdenamientoDeShell.html>

[https://www.ecured.cu/Algoritmo de Ordenamiento Shell](https://www.ecured.cu/Algoritmo_de_Ordenamiento_Shell)

<https://sites.google.com/site/jctovilla/programacion/c-c-para-linux-y-mac/ordenar-por-el-metodo-shell>